

Pagination avec Express.js & React.js

Qu'est-ce que la pagination ?

La **pagination** divise un grand nombre de données en plusieurs pages.

Exemple : si tu as **100 produits**, tu affiches **10 par page** → ça fait **10 pages**.

Importance :

- Améliore la **performance** (moins de données transférées).
 - Rend l'affichage plus **rapide et clair** pour l'utilisateur.
-

Backend : Express.js + MongoDB

Étape 1 : Route paginée

```
// routes/productRoutes.js

import express from "express";

import Product from "../models/Product.js";

const router = express.Router();

// GET /products?page=1&limit=10

router.get("/", async (req, res) => {

  try {

    // 1. Lire les paramètres

    const page = parseInt(req.query.page) || 1;

    const limit = parseInt(req.query.limit) || 10;

    const skip = (page - 1) * limit;
```

```
// 2. Récupérer les données

const products = await Product.find().skip(skip).limit(limit);


// 3. Compter le total

const total = await Product.countDocuments();


// 4. Réponse JSON

res.json({

  products,

  total,

  page,

  pages: Math.ceil(total / limit),

});

} catch (error) {

  res.status(500).json({ message: "Erreur serveur" });

}

});

export default router;
```

Explication :

- `page` → page actuelle (défaut = 1).
- `limit` → nombre d'éléments par page (défaut = 10).

- `skip` → combien d'éléments sauter avant de commencer.
 - On envoie au frontend : **les produits + infos de pagination (page, total, pages)**.
-

Frontend : React.js

Étape 1 : State pour stocker les données

```
const [products, setProducts] = useState([]);  
const [page, setPage] = useState(1);  
const [pages, setPages] = useState(1);
```

Étape 2 : Récupérer les données depuis l'API

```
useEffect(() => {  
  fetch(`http://localhost:5000/products?page=${page}&limit=5`)  
    .then((res) => res.json())  
    .then((data) => {  
      setProducts(data.products);  
      setPages(data.pages);  
    });  
}, [page]);
```

Explication :

- Chaque fois que `page` change, `useEffect` appelle l'API.
- On met à jour `products` et `pages`.

Étape 3 : Afficher les produits

```
<ul>

  {products.map((p) => (

    <li key={p._id} className="border-b py-2">{p.name}</li>

  ))}

</ul>
```

Explication : toujours mettre une **key** unique (**_id** de MongoDB).

Étape 4 : Boutons de pagination

```
<div className="flex gap-2 mt-4">

  <button

    disabled={page === 1}

    onClick={() => setPage(page - 1)}

    className="px-4 py-2 bg-gray-300 rounded"

  >

    Précédent

  </button>

  <span>Page {page} / {pages}</span>

  <button

    disabled={page === pages}
```

```
      onClick={() => setPage(page + 1)}

      className="px-4 py-2 bg-gray-300 rounded"

    >

      Suivant

    </button>

  </div>
```

Explication :

- Désactive "Précédent" si `page === 1`.
- Désactive "Suivant" si `page === pages`.

Exemple complet côté React

```
// components/ProductList.jsx

import React, { useEffect, useState } from "react";

const ProductList = () => {

  const [products, setProducts] = useState([]);

  const [page, setPage] = useState(1);

  const [pages, setPages] = useState(1);

  useEffect(() => {

    fetch(`http://localhost:5000/products?page=${page}&limit=5`)
```

```

        .then((res) => res.json())
        .then((data) => {
            setProducts(data.products);
            setPages(data.pages);
        });
    }, [page]);

    return (
        <div>
            <h1 className="text-xl font-bold mb-4">Liste des
Produits</h1>

            <ul className="mb-4">
                {products.map((p) => (
                    <li key={p._id} className="border-b py-2">
                        {p.name}
                    </li>
                ))}
            </ul>

            <div className="flex gap-2">
                <button
                    disabled={page === 1}
                    onClick={() => setPage(page - 1)}
                    className="px-4 py-2 bg-gray-300 rounded"

```

```

        >
        Précédent
    </button>

    <span>Page {page} / {pages}</span>

    <button
        disabled={page === pages}
        onClick={() => setPage(page + 1)}
        className="px-4 py-2 bg-gray-300 rounded"
    >
        Suivant
    </button>
</div>
</div>

);

};

export default ProductList;

```

Résumé des points importants

- Backend :

`skip()` → ignorer des documents.

`limit()` → limiter le nombre de résultats.

`countDocuments()` → connaître le total.

- **Frontend :**

`useState` → stocker `page` et `pages`.

`useEffect` → appeler l'API quand `page` change.

Boutons → naviguer entre les pages avec contrôle.